Plateforme Big Data & IA pour la supervision énergétique distribuée avec Spring Boot, MongoDB, Redis, Cassandra et Docker

Guide de réalisation pas à pas pour les étudiants

Fabrice RAZAFINDRAIBE
Projet de Master 1 MSI – Spring Boot – Big Data et Intelligence Artificielle

2026

Table des matières

Résumé **4**

1 Introduction générale **4**

2 Compétences visées **4**

3 Objectifs du projet **5**

3.1 Objectif général 5

3.2 Objectifs spécifiques 5

4 Périmètre pédagogique : ce qui est obligatoire et ce qui est optionnel **5**

4.1 Niveau 1 – MVP obligatoire 5

4.2 Niveau 2 – Extensions valorisées 6

5 Scénario fonctionnel du projet **6**

6 Architecture générale du système **7**

6.1 Rôle de chaque composant 7

7 Organisation recommandée du dépôt Git **8**

8 Préparation de l'environnement de travail **8**

8.1 Logiciels nécessaires 8

8.2 Travail demandé 8

9 Étape 1 – Concevoir le contrat de données **9**
9.1 Modèle minimal d'une mesure . . . 9
9.2 Exemple avec Jakarta Validation . . . 9
9.3 Questions à traiter dans le rapport . . . 9

10 Étape 2 – Développer le simulateur de capteurs **10**
10.1 Règle simple de génération . . . 10
10.2 Test demandé . . . 10

11 Étape 3 – Construire le Collector Service **11**
11.1 Endpoint minimal . . . 11
11.2 Pourquoi répondre HTTP 202 ? . . . 11
11.3 À réaliser . . . 11

12 Étape 4 – Mettre en place Redis Streams **12**
12.1 Travail demandé . . . 12
12.2 Expérience pédagogique . . . 12

13 Étape 5 – Modéliser Cassandra pour les séries temporelles **12**
13.1 Point pédagogique important . . . 13
13.2 Travail demandé . . . 13

14 Étape 6 – Utiliser MongoDB pour les données documentaires **13**
14.1 API minimale à développer . . . 14

15 Étape 7 – Développer le Storage Service **14**
15.1 Attention à l'idempotence . . . 15

16 Étape 8 – Exposer les données via une API de consultation **15**
16.1 Endpoints recommandés . . . 15
16.2 Documentation . . . 15

17 Étape 9 – Construire le module Analytics **16**
17.1 Niveau A – Analyse descriptive obligatoire . . . 16
17.2 Niveau B – Détection simple d'anomalie . . . 16
17.3 Niveau C – Prévision IA . . . 16
17.4 Métriques de prévision . . . 17

18 Exemple d'intégration d'un modèle IA avec DJL **17**

19 Étape 10 – Dockeriser la plateforme **17**
19.1 Exemple de fichier `docker-compose.yml` 18
19.2 Travail demandé 19

20 Étape 11 – Observabilité et supervision **19**
20.1 Métriques recommandées 19
20.2 Expérience demandée 20

21 Étape 12 – Sécurité et gestion des accès **20**

22 Stratégie de tests **20**
22.1 Tests unitaires 20
22.2 Tests d'intégration 21
22.3 Scénarios fonctionnels obligatoires 21
22.4 Tableau de preuve de tests 21

23 Plan de travail conseillé **21**

24 Répartition du travail en groupe **22**

25 Livrables attendus **22**

26 Structure recommandée du rapport final **23**

27 Points de contrôle avec l'enseignant **23**

28 Checklist avant remise **23**

29 Innovations et extensions possibles **24**

30 Conclusion **24**

Résumé

Ce projet tutoré propose de concevoir une plateforme intelligente de supervision énergétique capable de collecter, stocker, analyser et restituer des données de consommation issues de capteurs simulés ou réels. Il mobilise **Spring Boot**, **MongoDB**, **Redis**, **Cassandra**, **Docker**, ainsi qu'un module d'**intelligence artificielle** pour la prévision et la détection d'anomalies.

L'objectif pédagogique n'est pas seulement d'obtenir une application fonctionnelle. Les étudiants doivent comprendre **pourquoi chaque technologie est utilisée**, apprendre à construire progressivement une architecture distribuée, tester chaque composant de manière indépendante, documenter les choix techniques et démontrer la qualité du système final.

Le projet est donc organisé en étapes guidées. Chaque étape précise :

— l'objectif à atteindre ;
— les tâches à réaliser ;
— les fichiers ou composants à produire ;
— les tests à effectuer ;
— le livrable attendu ;
— le critère permettant de considérer l'étape comme validée.

1 Introduction générale

La transition énergétique et la généralisation des objets connectés produisent des volumes croissants de données : puissance instantanée, tension, courant, température, état d'un équipement, consommation cumulée, etc. Ces données sont générées en continu et doivent être traitées rapidement afin de permettre la supervision, l'alerte et l'aide à la décision.

Une application classique utilisant une seule base de données peut convenir à petite échelle, mais elle montre rapidement ses limites lorsqu'il faut gérer simultanément :

— un flux continu de mesures ;
— des lectures très fréquentes ;
— des historiques volumineux ;
— des besoins de cache ;
— des analyses prédictives ;
— une forte disponibilité des services.

Le présent projet demande donc aux étudiants de construire une plateforme distribuée appelée dans ce document **SmartEnergy Hub**. Elle devra assurer quatre fonctions principales : **collecter**, **stocker**, **analyser** et **exposer** les données énergétiques.

2 Compétences visées

A l'issue du projet, l'étudiant devra être capable de :

1. concevoir une architecture orientée services avec Spring Boot ;
2. développer et documenter des API REST ;
3. justifier l'usage de plusieurs bases NoSQL selon le besoin ;
4. manipuler un flux de données avec Redis Streams ;
5. modéliser des données temporelles dans Cassandra ;
6. stocker des documents métier dans MongoDB ;
7. conteneuriser une application avec Docker Compose ;
8. mettre en place des tests unitaires et d'intégration ;
9. exposer des métriques de supervision ;
10. intégrer un modèle simple de prévision ou de détection d'anomalie ;
11. présenter les choix d'architecture, les résultats et les limites du système.

3 Objectifs du projet

3.1 Objectif général

Développer une plateforme distribuée de gestion énergétique intégrant les principes du Big Data, de l'Intelligence Artificielle et du DevOps.

3.2 Objectifs spécifiques

1. Intégrer plusieurs bases NoSQL selon leurs rôles :
   — **MongoDB** pour les équipements, les agrégats, les rapports et les données documentaires ;
   — **Redis** pour le cache et le transit rapide des mesures ;
   — **Cassandra** pour les séries temporelles volumineuses.
2. Implémenter un backend modulaire ou microservices avec **Spring Boot**.
3. Intégrer un module d'**analyse prédictive**.
4. Assurer la **résilience**, l'observabilité et la capacité d'évolution du système.
5. Déployer l'ensemble avec **Docker Compose**.

4 Périmètre pédagogique : ce qui est obligatoire et ce qui est optionnel

Pour éviter qu'un projet trop ambitieux ne devienne impossible à terminer, la réalisation est divisée en deux niveaux.

4.1 Niveau 1 – MVP obligatoire

Le MVP (Minimum Viable Product) doit obligatoirement comporter :

— un simulateur de capteurs ;
— un service Spring Boot de collecte ;
— Redis Streams pour transporter les mesures ;
— un service de stockage ;
— Cassandra pour les mesures temporelles ;
— MongoDB pour les équipements et les agrégats ;
— une API REST de consultation ;
— un calcul simple de consommation moyenne et de détection de dépassement ;
— Docker Compose ;
— une documentation Swagger/OpenAPI ;
— des tests démontrant que la chaîne complète fonctionne.

4.2 Niveau 2 – Extensions valorisées

Les fonctionnalités suivantes peuvent être ajoutées après validation du MVP :

— modèle LSTM ou modèle de régression pour la prévision ;
— API Gateway ;
— authentification JWT ;
— Prometheus et Grafana ;
— gestion automatique des alertes ;
— Kafka à la place ou en complément de Redis Streams ;
— déploiement Kubernetes ;
— interface Web ou mobile.

5 Scénario fonctionnel du projet

On considère un ensemble de bâtiments contenant plusieurs compteurs ou capteurs énergétiques. Chaque capteur envoie périodiquement une mesure.

Une mesure contient au minimum :

— l'identifiant du capteur ;
— la date et l'heure de mesure ;
— la puissance consommée en kW ;
— la tension en V ;
— la température ambiante en °C ;
— éventuellement le courant, la fréquence ou un état de fonctionnement.

Exemple de mesure JSON :

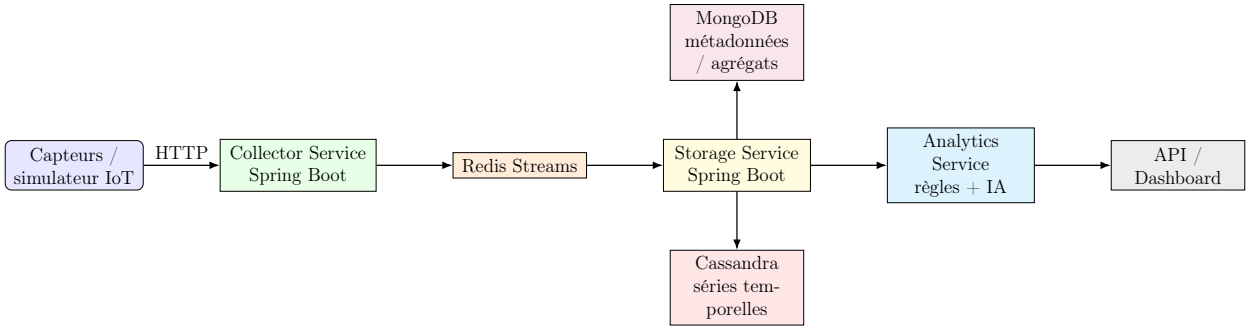
```
{
  "sensorId": "SENSOR-001",
  "timestamp": "2026-09-30T08:15:00Z",
```

```
  "powerKw": 3.72,
  "voltage": 226.4,
  "temperature": 29.1
}
```

Le système devra pouvoir répondre à des questions telles que :

— Quelle est la dernière valeur reçue pour un capteur ?
— Quelle a été sa consommation moyenne entre deux dates ?
— Quels capteurs dépassent un seuil donné ?
— Quelle consommation est prévue pour les prochaines heures ?
— Quels événements sont considérés comme anormaux ?

6 Architecture générale du système



6.1 Rôle de chaque composant

Composant	Rôle	Question à laquelle il répond
Simulateur IoT	Générer des mesures réalistes à intervalle régulier	Comment tester sans disposer de vrais capteurs ?
Collector Service	Valider et recevoir les mesures	Comment protéger le système contre les données invalides ?
Redis Streams	Tamponner et distribuer rapidement les événements	Comment découpler la réception du stockage ?
Storage Service	Consommer les flux et persister les mesures	Comment centraliser la logique de persistance ?
Cassandra	Stocker de grands volumes de mesures temporelles	Comment lire rapidement l'historique d'un capteur ?
MongoDB	Stocker capteurs, configurations, agrégats et rapports	Comment conserver des documents évolutifs ?
Analytics Service	Calculer indicateurs, anomalies et prédictions	Comment transformer des mesures en information utile ?
API / Dashboard	Fournir les résultats aux utilisateurs	Comment rendre les données consultables ?

7 Organisation recommandée du dépôt Git

Chaque groupe doit créer un dépôt versionné. Une organisation possible est :

```
smartenergy-hub/
|-- README.md
|-- docker-compose.yml
|-- docs/
|   |-- architecture.md
|   |-- api.md
|   `-- tests.md
|-- simulator/
|-- collector-service/
|-- storage-service/
|-- analytics-service/
|-- gateway-service/          # optionnel
|-- monitoring/               # optionnel
|-- scripts/
|   |-- init-cassandra.cql
|   `-- sample-data.json
`-- models/                   # modèles IA exportés
```

Le fichier `README.md` doit expliquer comment lancer le projet avec une commande unique lorsque cela est possible.

8 Préparation de l'environnement de travail

Objectif de l'étape : Obtenir un environnement reproductible identique pour tous les membres du groupe.

8.1 Logiciels nécessaires

Chaque groupe doit disposer au minimum de :

- JDK 17 ou version LTS compatible avec le projet ;
- Maven ou Gradle ;
- Docker et Docker Compose ;
- Git ;
- un IDE Java ;
- Postman, Bruno ou un outil équivalent ;
- un navigateur pour Swagger UI.

8.2 Travail demandé

1. créer le dépôt Git ;

2. créer une branche de développement ;
3. générer un premier projet Spring Boot avec Spring Web et Actuator ;
4. créer une route `/api/health` renvoyant un message simple ;
5. construire le projet ;
6. lancer l'application et vérifier le port HTTP.

Livrable attendu : Capture ou démonstration de l'API de santé, dépôt Git initialisé et README contenant les prérequis.

Critère de validation : La commande de construction fonctionne et l'URL de santé répond avec un code HTTP 200.

9 Étape 1 – Concevoir le contrat de données

Objectif de l'étape : Définir précisément les données échangées avant de programmer la logique métier.

9.1 Modèle minimal d'une mesure

```
public record EnergyMeasurementRequest(
    String sensorId,
    Instant timestamp,
    Double powerKw,
    Double voltage,
    Double temperature
) {}
```

Les étudiants doivent ajouter des contraintes de validation : identifiant obligatoire, puissance positive ou nulle, tension dans une plage plausible, etc.

9.2 Exemple avec Jakarta Validation

```
public record EnergyMeasurementRequest(
    @NotBlank String sensorId,
    @NotNull Instant timestamp,
    @NotNull @PositiveOrZero Double powerKw,
    @NotNull @DecimalMin("0.0") Double voltage,
    Double temperature
) {}
```

9.3 Questions à traiter dans le rapport

— Pourquoi utiliser un horodatage UTC ?

— Que faire lorsqu'un capteur envoie deux fois la même mesure ?
— Faut-il accepter une mesure très ancienne ?
— Quelle différence existe entre une erreur de format et une anomalie énergétique ?

Livrable attendu : Schéma JSON des mesures, DTO Java et règles de validation documentées.

Critère de validation : Une mesure correcte est acceptée et une mesure incorrecte provoque une réponse HTTP 400 avec un message compréhensible.

10 Étape 2 – Développer le simulateur de capteurs

Objectif de l'étape : Créer une source de données contrôlée permettant de tester la plateforme sans matériel IoT réel.

Le simulateur peut être écrit en Java, Python ou JavaScript. Il doit générer plusieurs capteurs et envoyer une mesure toutes les quelques secondes vers le Collector Service.

10.1 Règle simple de génération

On peut simuler la puissance par :

$$P_t = P_{base} + A \sin\left(\frac{2\pi t}{T}\right) + \epsilon_t \tag{1}$$

avec :

— P_{base} : consommation moyenne ;
— A : amplitude des variations ;
— T : période ;
— ϵ_t : bruit aléatoire.

Le simulateur doit également pouvoir injecter volontairement une anomalie, par exemple une puissance multipliée par trois pendant une courte période.

10.2 Test demandé

Lancer 5 capteurs simulés pendant au moins 2 minutes et vérifier le nombre de requêtes reçues.

Livrable attendu : Code du simulateur, paramètres configurables et exemple d'exécution.

Critère de validation : Le simulateur génère automatiquement des données valides et permet d'activer au moins un scénario d'anomalie.

11 Étape 3 – Construire le Collector Service

Objectif de l'étape : Recevoir les mesures, les valider et les publier dans Redis Streams sans bloquer l'émetteur.

11.1 Endpoint minimal

```java
@RestController
@RequestMapping("/api/measurements")
public class MeasurementController {

    private final MeasurementPublisher publisher;

    public MeasurementController(MeasurementPublisher publisher) {
        this.publisher = publisher;
    }

    @PostMapping
    public ResponseEntity<Void> receive(
            @Valid @RequestBody EnergyMeasurementRequest request) {
        publisher.publish(request);
        return ResponseEntity.accepted().build();
    }
}
```

11.2 Pourquoi répondre HTTP 202 ?

La réception et le stockage final sont découplés. Le Collector confirme que la donnée est acceptée pour traitement, mais il ne garantit pas encore qu'elle a été écrite dans toutes les bases.

11.3 À réaliser

1. créer le DTO de requête ;
2. valider le JSON ;
3. créer un service de publication ;
4. publier chaque événement dans un stream Redis nommé par exemple `energy-measurements` ;
5. journaliser l'identifiant du capteur et l'identifiant du message Redis ;
6. prévoir une gestion d'erreur simple.

Livrable attendu : Collector Service exécutable et collection de tests HTTP.

Critère de validation : Une requête valide crée effectivement un événement dans Redis ; une requête invalide n'en crée aucun.

12 Étape 4 – Mettre en place Redis Streams

Objectif de l'étape : Utiliser Redis comme couche de transit rapide entre réception et persistance.

Chaque événement peut être enregistré sous forme de champs : `sensorId`, `timestamp`, `powerKw`, `voltage`, `temperature`.

Les étudiants doivent comprendre les notions suivantes :

- stream ;
- identifiant d'événement ;
- consumer group ;
- consumer ;
- accusé de traitement ;
- message en attente.

12.1 Travail demandé

Créer un consumer group `storage-group`. Le Storage Service consommera les événements de ce groupe et les accusera seulement après persistance réussie.

12.2 Expérience pédagogique

Arrêter volontairement le Storage Service pendant que le simulateur fonctionne. Les étudiants doivent observer que les événements continuent à entrer dans Redis. Relancer ensuite le service et vérifier qu'il reprend le traitement.

Livrable attendu : Configuration Redis, consumer group et preuve de reprise après interruption.

Critère de validation : Aucune mesure produite pendant l'arrêt du Storage Service ne doit disparaître dans le scénario de test prévu.

13 Étape 5 – Modéliser Cassandra pour les séries temporelles

Objectif de l'étape : Concevoir une table Cassandra à partir des requêtes à effectuer, et non comme une base relationnelle classique.

Une requête principale est : « récupérer l'historique d'un capteur entre deux dates ».

Une table possible est :

```
CREATE KEYSPACE IF NOT EXISTS smartenergy
WITH replication = {
```

```
  'class': 'SimpleStrategy',
  'replication_factor': 1
};

CREATE TABLE IF NOT EXISTS smartenergy.measurements_by_sensor_day (
  sensor_id text,
  day date,
  measured_at timestamp,
  power_kw double,
  voltage double,
  temperature double,
  PRIMARY KEY ((sensor_id, day), measured_at)
) WITH CLUSTERING ORDER BY (measured_at DESC);
```

13.1 Point pédagogique important

Le couple `(sensor_id, day)` forme ici une clé de partition. Cette stratégie évite qu'un capteur très actif accumule toutes ses mesures dans une seule partition pendant plusieurs années.

13.2 Travail demandé

1. créer le keyspace ;
2. créer la table ;
3. implémenter l'entité Spring Data Cassandra ;
4. écrire une mesure ;
5. lire les mesures d'un capteur pour une journée ;
6. mesurer approximativement le temps de réponse sur un jeu de données plus important.

Livrable attendu : Script CQL, entité Java, repository et test de lecture d'historique.

Critère de validation : Le groupe sait expliquer le choix de la clé de partition et démontrer la lecture de données temporelles.

14 Étape 6 – Utiliser MongoDB pour les données documentaires

Objectif de l'étape : Stocker des informations qui n'ont pas la même structure ni le même usage que les mesures brutes.

Un document de capteur peut être représenté ainsi :

```
{
  "sensorId": "SENSOR-001",
  "name": "Compteur salle serveur",
```

```
  "building": "Bâtiment A",
  "floor": 1,
  "maxPowerKw": 8.0,
  "active": true,
  "tags": ["critical", "server-room"]
}
```

MongoDB peut également stocker un agrégat quotidien :

```
{
  "sensorId": "SENSOR-001",
  "date": "2026-09-30",
  "avgPowerKw": 3.42,
  "maxPowerKw": 7.81,
  "energyKwh": 68.2,
  "anomalyCount": 2
}
```

14.1 API minimale à développer

- `POST /api/sensors` : créer un capteur ;
- `GET /api/sensors` : lister les capteurs ;
- `GET /api/sensors/{id}` : afficher un capteur ;
- `PUT /api/sensors/{id}` : modifier un capteur ;
- `GET /api/sensors/{id}/daily-summary` : obtenir un résumé.

Livrable attendu : Collections MongoDB, repositories et endpoints de consultation.

Critère de validation : Les métadonnées sont consultables indépendamment des mesures brutes stockées dans Cassandra.

15 Étape 7 – Développer le Storage Service

Objectif de l'étape : Consommer les messages Redis, les convertir en objets métier et les persister de façon fiable.

Le traitement attendu est :

1. lire un événement Redis ;
2. vérifier sa structure ;
3. enregistrer la mesure dans Cassandra ;
4. mettre à jour si nécessaire un cache ou un agrégat ;
5. accuser le message Redis ;
6. journaliser les erreurs sans arrêter tout le consumer.

15.1 Attention à l'idempotence

Un même événement peut parfois être traité deux fois. Les étudiants doivent proposer une stratégie pour éviter que cela ne fausse les résultats : identifiant unique d'événement, contrôle applicatif ou conception tolérante aux doublons.

Livrable attendu : Storage Service complet avec consumer Redis et persistance Cassandra/MongoDB.

Critère de validation : Une mesure envoyée au Collector est retrouvée dans Cassandra et les informations attendues sont mises à jour dans MongoDB.

16 Étape 8 – Exposer les données via une API de consultation

Objectif de l'étape : Rendre les données exploitables par un utilisateur ou un futur dashboard.

16.1 Endpoints recommandés

Endpoint	Méthode	Résultat attendu
`/api/sensors`	GET	Liste des capteurs
`/api/sensors/{id}`	GET	Métadonnées du capteur
`/api/measurements/{sensorId}`	GET	Historique filtré par dates
`/api/measurements/{sensorId}/latest`	GET	Dernière mesure connue
`/api/analytics/{sensorId}/summary`	GET	Moyenne, maximum, énergie estimée
`/api/analytics/{sensorId}/anomalies`	GET	Liste des anomalies détectées
`/api/analytics/{sensorId}/forecast`	GET	Prévisions si le module IA est actif

Les paramètres `from` et `to` doivent être utilisés pour limiter les périodes demandées.

16.2 Documentation

Tous les endpoints doivent apparaître dans une documentation OpenAPI/Swagger avec exemples de requêtes et de réponses.

Livrable attendu : API de lecture documentée et testable depuis Swagger UI.

Critère de validation : Un utilisateur ne connaissant pas le code peut comprendre et tester les principales routes grâce à la documentation.

17 Étape 9 – Construire le module Analytics

Objectif de l'étape : Transformer les mesures brutes en indicateurs et introduire progressivement l'intelligence artificielle.

17.1 Niveau A – Analyse descriptive obligatoire

Avant toute IA, le groupe doit calculer :

- moyenne de puissance ;
- valeur maximale ;
- minimum ;
- estimation de l'énergie consommée ;
- nombre de mesures reçues ;
- nombre de dépassements d'un seuil.

Pour des mesures espacées de Δt heures, une estimation simple de l'énergie est :

$$E \approx \sum_{i=1}^{n} P_i \Delta t \tag{2}$$

avec E en kWh si P_i est exprimée en kW.

17.2 Niveau B – Détection simple d'anomalie

Une première méthode peut utiliser un seuil défini pour chaque capteur :

$$Anomalie = P_t > P_{max} \tag{3}$$

Une méthode statistique peut utiliser un score standardisé :

$$z_t = \frac{P_t - \mu}{\sigma} \tag{4}$$

Une mesure peut être signalée lorsque $|z_t|$ dépasse un seuil choisi et justifié.

17.3 Niveau C – Prévision IA

Après validation des niveaux précédents, le groupe peut entraîner un modèle de régression, un réseau dense ou un LSTM. La démarche doit être clairement séparée en :

1. préparation des données ;
2. séparation entraînement / validation / test ;
3. entraînement ;
4. mesure de l'erreur ;
5. export du modèle ;
6. intégration dans l'application.

17.4 Métriques de prévision

Le groupe doit utiliser au moins une métrique telle que :

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{5}$$

ou :

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{6}$$

Livrable attendu : Service d'analyse avec indicateurs obligatoires ; module IA documenté si réalisé.

Critère de validation : Les étudiants démontrent au minimum un calcul descriptif correct et expliquent les limites de la méthode de détection/prévision choisie.

18 Exemple d'intégration d'un modèle IA avec DJL

Un modèle entraîné en dehors de Spring Boot peut être exporté, par exemple au format ONNX, puis chargé par un service Java.

```
Criteria<float[], float[]> criteria = Criteria.builder()
    .setTypes(float[].class, float[].class)
    .optModelPath(Paths.get("models/energy.onnx"))
    .build();

try (ZooModel<float[], float[]> model = ModelZoo.loadModel(criteria);
     Predictor<float[], float[]> predictor = model.newPredictor()) {

    float[] prediction = predictor.predict(inputArray);
}
```

Dans le rapport, il faut préciser :

- la taille de la fenêtre temporelle d'entrée ;
- les variables utilisées ;
- le traitement des valeurs manquantes ;
- la méthode de normalisation ;
- les métriques d'évaluation ;
- les limites du modèle.

19 Étape 10 – Dockeriser la plateforme

Objectif de l'étape : Permettre à un autre étudiant ou à l'enseignant de lancer tout le système de manière reproductible.

Chaque service Spring Boot doit posséder un `Dockerfile`. Exemple :

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

19.1 Exemple de fichier `docker-compose.yml`

```
services:
  mongo:
    image: mongo:7
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

  redis:
    image: redis:7
    ports:
      - "6379:6379"

  cassandra:
    image: cassandra:5
    ports:
      - "9042:9042"
    volumes:
      - cassandra_data:/var/lib/cassandra

  collector-service:
    build: ./collector-service
    ports:
      - "8081:8080"
    depends_on:
      - redis

  storage-service:
    build: ./storage-service
    ports:
      - "8082:8080"
    depends_on:
      - redis
      - cassandra
      - mongo

  analytics-service:
    build: ./analytics-service
    ports:
      - "8083:8080"
    depends_on:
```

```
      - cassandra
      - mongo

volumes:
  mongo_data:
  cassandra_data:
```

19.2 Travail demandé

1. construire tous les JAR ;
2. construire les images ;
3. lancer `docker compose up -d` ;
4. vérifier l'état des conteneurs ;
5. lancer le simulateur ;
6. interroger l'API ;
7. arrêter et relancer la plateforme en vérifiant la persistance des données.

Livrable attendu : Dockerfiles, fichier Compose et procédure de démarrage dans le README.

Critère de validation : Le projet complet peut être lancé sur une autre machine avec un nombre minimal de commandes documentées.

20 Étape 11 – Observabilité et supervision

Objectif de l'étape : Rendre visible l'état du système et mesurer son comportement.

Chaque service Spring Boot doit activer Actuator. Les groupes peuvent ensuite ajouter Prometheus et Grafana.

20.1 Métriques recommandées

— nombre de mesures reçues ;
— nombre de mesures rejetées ;
— temps moyen de traitement ;
— nombre de messages Redis en attente ;
— nombre d'erreurs de persistance ;
— latence de prédiction ;
— disponibilité de chaque service.

20.2 Expérience demandée

Générer une charge plus importante et observer l'évolution des temps de réponse. Il n'est pas demandé de faire un benchmark scientifique complet, mais le groupe doit être capable de commenter les résultats.

Livrable attendu : Endpoints Actuator fonctionnels et, en extension, tableau de bord Grafana.

Critère de validation : Le groupe peut montrer l'état de santé du système et au moins trois métriques utiles.

21 Étape 12 – Sécurité et gestion des accès

Objectif de l'étape : Comprendre les mécanismes de protection d'une API distribuée.

Le niveau minimal consiste à :

- ne pas exposer de mots de passe dans Git ;
- utiliser des variables d'environnement ;
- valider les entrées ;
- ne pas exposer inutilement les ports des bases en production ;
- définir des profils de configuration.

En extension, les groupes peuvent ajouter :

- Spring Security ;
- authentification JWT ;
- rôles `ADMIN` et `VIEWER` ;
- OAuth2 ;
- HTTPS via un reverse proxy.

Livrable attendu : Note de sécurité indiquant les secrets à protéger, les accès exposés et les mécanismes mis en place.

Critère de validation : Aucun secret réel ne doit être commité dans le dépôt et les endpoints sensibles doivent être identifiés.

22 Stratégie de tests

Les tests ne doivent pas être reportés à la fin du projet. Chaque étape doit être accompagnée d'une preuve de fonctionnement.

22.1 Tests unitaires

Ils concernent notamment :

— les calculs d'énergie ;
— la détection d'anomalies ;
— la validation des objets ;
— les transformations entre DTO et entités.

22.2 Tests d'intégration

Ils doivent vérifier au moins :

— API → Redis ;
— Redis → Storage Service ;
— Storage Service → Cassandra ;
— Storage Service → MongoDB ;
— lecture via API.

22.3 Scénarios fonctionnels obligatoires

1. envoyer une mesure valide ;
2. envoyer une mesure invalide ;
3. rechercher l'historique d'un capteur ;
4. simuler une anomalie ;
5. arrêter temporairement un service ;
6. redémarrer le système ;
7. vérifier la persistance des données.

22.4 Tableau de preuve de tests

N°	Scénario	Résultat attendu	Résultat obtenu
1	Envoi d'une mesure valide	HTTP 202 + événement Redis	À compléter
2	Mesure sans `sensorId`	HTTP 400	À compléter
3	Lecture historique	Liste ordonnée de mesures	À compléter
4	Injection d'un pic	Anomalie détectée	À compléter
5	Arrêt Storage Service	Messages conservés dans Redis	À compléter
6	Redémarrage	Reprise du traitement	À compléter

23 Plan de travail conseillé

Le calendrier ci-dessous peut être adapté au volume horaire disponible.

Période	Travaux principaux	Point de contrôle
J1	Analyse, Git, architecture, contrat JSON	Architecture validée
J2	Simulateur + Collector Service	Données reçues et validées
J3	Redis Streams + consumer	Flux asynchrone fonctionnel
J4	Cassandra + MongoDB	Persistance démontrée
J5	API de consultation + Swagger	API testable
J	Analytics descriptif + anomalies	Indicateurs validés
J7	Docker Compose + tests d'intégration	MVP exécutable
J8	IA / sécurité / monitoring selon avancement	Extension documentée
J9	Tests finaux, rapport, corrections	Version candidate
J10	Soutenance et démonstration	Livraison finale

24 Répartition du travail en groupe

Pour un groupe de quatre étudiants, une répartition indicative est :

— **Étudiant A :** Collector, validation, Redis producer ;
— **Étudiant B :** Storage Service, Cassandra ;
— **Étudiant C :** MongoDB, API de consultation ;
— **Étudiant D :** Analytics, tests et monitoring.

Cependant, chaque membre doit connaître l'architecture complète. Les rôles ne doivent pas devenir des silos. Les revues de code croisées sont recommandées.

25 Livrables attendus

Le groupe doit remettre :

1. le dépôt contenant tout le code source ;
2. le fichier `docker-compose.yml` ;
3. les scripts d'initialisation des bases ;
4. les données ou le simulateur nécessaires à la démonstration ;
5. la documentation OpenAPI ;
6. le rapport technique ;
7. les preuves de tests ;
8. le modèle IA si cette extension est réalisée ;
9. une présentation de soutenance ;
10. une démonstration fonctionnelle de bout en bout.

26 Structure recommandée du rapport final

1. **Introduction et contexte**
2. **Cahier des charges et besoins fonctionnels**
3. **Architecture générale et justification des choix**
4. **Modélisation des données**
5. **Conception et réalisation des services**
6. **Gestion du flux Redis**
7. **Persistance Cassandra et MongoDB**
8. **Module Analytics et IA**
9. **Conteneurisation et déploiement**
10. **Tests et résultats**
11. **Sécurité, limites et difficultés rencontrées**
12. **Améliorations possibles**
13. **Conclusion**

Le rapport ne doit pas être une simple copie du code. Il doit expliquer les décisions, comparer les solutions possibles et commenter les résultats obtenus.

27 Points de contrôle avec l'enseignant

Il est recommandé d'organiser au moins quatre validations intermédiaires :

1. **Validation 1 :** architecture, contrat JSON et organisation Git ;
2. **Validation 2 :** chaîne Collector → Redis → Storage ;
3. **Validation 3 :** persistance Cassandra/MongoDB et API ;
4. **Validation 4 :** Docker, analytics, tests et préparation de la démonstration.

Chaque validation doit être accompagnée d'une démonstration courte et non uniquement d'une présentation théorique.

28 Checklist avant remise

Avant la remise finale, le groupe doit pouvoir répondre « oui » aux questions suivantes :

— Le projet se compile-t-il sans modification manuelle ?
— Le fichier README explique-t-il clairement le démarrage ?
— Docker Compose démarre-t-il les dépendances ?
— Le simulateur produit-il des données automatiquement ?
— Les requêtes invalides sont-elles rejetées correctement ?

— Les messages sont-ils visibles dans Redis ?
— Les mesures sont-elles persistées dans Cassandra ?
— Les capteurs et agrégats sont-ils disponibles dans MongoDB ?
— L'API permet-elle de lire l'historique ?
— Les résultats analytiques sont-ils vérifiables ?
— Les principaux scénarios de panne ont-ils été testés ?
— Les secrets sont-ils absents du dépôt ?
— Swagger est-il accessible ?
— Le groupe sait-il expliquer ses choix sans lire le rapport ?

29 Innovations et extensions possibles

Une fois le MVP validé, les étudiants peuvent proposer :

1. un assistant conversationnel permettant d'interroger les indicateurs énergétiques en langage naturel ;
2. la génération automatique de rapports ;
3. un système de notifications lors d'une anomalie ;
4. Kafka pour l'ingestion à plus grande échelle ;
5. Kubernetes pour l'orchestration ;
6. une interface Web temps réel ;
7. un système multi-bâtiments et multi-utilisateurs ;
8. une comparaison entre plusieurs modèles de prévision.

Toute extension doit apporter une valeur démontrable. Ajouter une technologie uniquement pour augmenter le nombre d'outils utilisés n'est pas un objectif en soi.

30 Conclusion

SmartEnergy Hub constitue un projet intégrateur permettant aux étudiants de mobiliser simultanément le développement backend, les bases NoSQL, le traitement de flux, le Big Data, l'intelligence artificielle et le déploiement conteneurisé.

La démarche proposée impose une progression : construire d'abord un système simple et vérifiable, puis introduire progressivement la distribution, l'analyse et l'IA. Cette organisation permet d'éviter qu'une difficulté sur une technologie bloque l'ensemble du projet et aide les étudiants à identifier précisément l'origine d'un problème.

La réussite du projet ne sera donc pas jugée seulement sur le nombre de technologies employées, mais sur la **cohérence de l'architecture**, la **qualité de l'implémentation**, la **capacité à tester**, la **justification des choix** et la **maîtrise démontrée lors de la soutenance**.